SHERLOCK

Security Review For Delpho



Collaborative Audit Prepared For: **Delpho**
Lead Security Expert(s): **eeyore**
TessKimy
Date Audited: **September 3 - September 8, 2026**

Introduction

Delpho is building the productive balance-sheet layer of Hyperliquid by combining a purpose-built CDP with a yield-bearing stablecoin, which sources its yield from delta-neutral positions automated fully on Hyperliquid.

Delpho lets users mint against their crypto collateral and stablecoins at a 0% borrow cost while retaining directionality to their assets and earning composable, best-in-class yield.

Scope

Repository: Delpho-labs/delpho_contracts

Audited Commit: 68fcab60e25381f682e4f1fb39346c65b50dd2cc

Final Commit: 8bc0ee1c785baae28adee37cdcab97821a74e887

Files:

- src/base/DelphoConfigurator.sol
- src/configurators/base/BaseConfig.sol
- src/configurators/base/DelphoExecutorConfig.sol
- src/configurators/base/DelphoHyperCoreConfig.sol
- src/configurators/base/DelphoPSMConfig.sol
- src/configurators/base/DelphoRedemptionConfig.sol
- src/configurators/base/DelphoVaultConfig.sol
- src/configurators/base/interfaces/IBaseConfig.sol
- src/configurators/base/interfaces/IDelphoExecutorConfig.sol
- src/configurators/base/interfaces/IDelphoHyperCoreConfig.sol
- src/configurators/base/interfaces/IDelphoPSMConfig.sol
- src/configurators/base/interfaces/IDelphoRedemptionConfig.sol
- src/configurators/base/interfaces/IDelphoVaultConfig.sol
- src/configurators/DelphoConfigProvider.sol
- src/configurators/interfaces/IDelphoConfigProvider.sol
- src/core/DelphoPSM.sol
- src/core/DelphoStabilityPoolV2.sol
- src/core/interfaces/IDelphoPSM.sol
- src/DelphoStakerV2.sol

- src/executor/base/DelphoBaseExecutor.sol
- src/executor/base/DelphoExecutorCoreExtension.sol
- src/executor/base/DelphoExecutorFeeExtension.sol
- src/executor/base/DelphoExecutorRecoveryExtension.sol
- src/executor/base/DelphoExecutorSwapExtension.sol
- src/executor/base/interfaces/IDelphoExecutorBase.sol
- src/executor/DelphoStablePortfolioMarginExecutor.sol
- src/executor/DelphoWhypePMExecutor.sol
- src/interfaces/IDelphoRouter.sol
- src/interfaces/IDelphoRouterV2.sol
- src/interfaces/IDelphoStabilityPool.sol
- src/interfaces/IDelphoStabilityPoolV2.sol
- src/periphery/DelphoRouter.sol
- src/periphery/DelphoRouterV2.sol
- src/types/DelphoTypes.sol

Final Commit Hash

8bc0ee1c785baae28adee37cdcab97821a74e887

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
1	3	8

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue H-1: Registered-but-disabled collateral is dropped by the config replay, freezing every position that holds it [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/issues/66

Summary

The migration replays the token registry by iterating `getAllSupportedTokens()`, which filters on the per-token `enabled` flag. WHYPE and KHYPE are registered-but-disabled on the live provider, so their `TokenConfig` is never carried to the new one.

`DelphoOracle` does not read `enabled`. It gates on `decimals == 0 || priceFeed == address(0)`, so both tokens price normally today and back live USDV debt. After the repoint their config is gone, the oracle reverts, and every position holding either token becomes unwithdrawable and unliquidatable.

Vulnerability Detail

`BaseConfigProviderUpgrade._migrateAllConfig` seeds from `oldConfig.getAllSupportedTokens()`. That getter filters on `enabled`, returning `[USDT, USDC]` on chain while `getTokenCount()` returns `4`. The token/oracle and deposit-limit loops iterate only that array and contain the only `addToken`, `setTokenPriceBounds` and `setTokenStalenessAge` call sites in the prod upgrade tree. WHYPE and KHYPE are therefore never registered on the new provider.

The oracle never agreed to that reading of `enabled`:

```
if (tokenConfig.decimals == 0 || tokenConfig.priceFeed == address(0)) revert
↪  DO_UNSUPPORTED_TOKEN();
```

Valuation also ignores the provider registry. `VaultMath._getUserCollateralTokensAndAmounts` builds its token array from `depositedTokens[user]`, the Vault's own set, which is cleared only when a balance reaches zero. Nothing removes a token from it on deregistration. So the Vault still treats WHYPE as collateral while the provider no longer has the feed to price it.

Verified against mainnet: `getTokenConfig(WHYPE)` returns `(0x2b25BC41…, false, 18, 18)`, `oracle.getTokenPrice(WHYPE)` returns `84244695`, and an unregistered token reverts `0x74387a2d`, which is `DO_UNSUPPORTED_TOKEN()`. The repointed stable Vault holds `totalCollateral(WHYPE) = 2166000000000000000`. Two affected holders, both with live debt and both in the sorted-debt list:

Address	Collateral	usdvMinted	getUserTotal
0xe6183440c540412F1459E6B98bC85ECb17cD21fA	340 USDC + 0.1 WHYPE	207260000	348390469
0xd2d66763C2E3f23044D5687058B3d38907C3E3C0	2.0 WHYPE only	50000000	168489390

`2 × 84244695 == 168489390` exactly, proving WHYPE is priced into live LTV.

Impact

`requestWithdrawal`, `claimWithdrawal`, `cancelWithdrawalRequest`, `collateralDeposit`, `mintStable` and liquidation all revert for affected addresses. Only `repayStable` still works. `_validateUserLTV` prices before checking debt, so debt-free depositors are frozen too, and LTV covers the whole position, so unaffected collateral is trapped alongside it.

Positions become unliquidatable, converting any HYPE drawdown into protocol bad debt. USDV fallback redemption also reverts for every redeemer, because it calls `getUserCollateralDetails` per borrower with no error handling and both holders are in the list it iterates.

The failure is deterministic and occurs the moment the Vault is repointed. `isTokenSupported(WHYPE)` returns `false` before and after, so the divergence is invisible to an operator checking that getter.

Code Snippet

https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/blob/4e11694f7a029115f3fdb01bd98b6425744c4bd3/delpho_contracts/script/prod/upgrade/base/BaseConfigProviderUpgrade.s.sol#L53

https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/blob/4e11694f7a029115f3fdb01bd98b6425744c4bd3/delpho_contracts/src/configurators/base/DelphoTokenConfig.sol#L61

https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/blob/4e11694f7a029115f3fdb01bd98b6425744c4bd3/delpho_contracts/src/core/DelphoOracle.sol#L162

https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/blob/4e11694f7a029115f3fdb01bd98b6425744c4bd3/delpho_contracts/src/libs/VaultMath.sol#L181

Tool Used

Manual Review.

Recommendation

Replay the full registry instead of the filtered view, then restore the disabled state:

```
-address[] memory tokens = oldConfig.getAllSupportedTokens();
+uint256 count = oldConfig.getTokenCount();
+address[] memory tokens = new address[](count);
+for (uint256 i; i < count; ++i) tokens[i] = oldConfig.supportedTokens(i);
```

For each token copy the feed, price bounds, staleness age and deposit limits, then call `updateToken(token, false, feed)` for any that were disabled. Assert that the new provider's `getTokenCount()` matches the old one. No contract change is required, and the fix must land before cutover.

Issue M-1: V2 applies isTokenSupported on withdrawal paths, so disabling a collateral permanently locks it in the WHYPE/KHYPE market [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/issues/67

Summary

`DelphoRouterV2` validates every supplied token against the market's ConfigProvider on three **exit** paths: `requestWithdrawal`, `cancelWithdrawalRequest` and `repayAndWithdraw`. `DelphoRouter` gates only the **entry** path.

That asymmetry in V1 is deliberate and correct: disabling a collateral should stop new deposits without orphaning existing positions. V2 breaks it. Once a collateral is disabled in the WHYPE market, holders cannot withdraw it, cancel a pending request, or close their position, and no V1 selector reaches that market's Vault to work around it.

Vulnerability Detail

V1 contains exactly one `isTokenSupported` call, inside `_handleDeposit`. Its withdrawal functions perform no token validation, and neither does `DelphoVault` on `requestWithdrawal`, `cancelWithdrawalRequest` or `claimWithdrawal`.

V2 adds the check to three exit paths.

`withdrawCollateral`, `redeem` and `repayStable` retain V1 parity, so the regression is confined to these three.

`_validateMarketToken` reverts `DR_UNSUPPORTED_TOKEN` when `config.isTokenSupported(token)` is false, and `isTokenSupported` returns the per-token `enabled` flag. So a routine `updateToken(token, false, feed)` immediately blocks all three paths for every holder of that collateral.

Inherited V1 selectors resolve through `delphoConfigProvider`, which points at the stable provider and therefore the stable Vault. A depositor in the WHYPE market has no reachable path to that market's Vault other than the V2 overloads, so there is no equivalent of the workaround that keeps the stable market's disabled collateral accessible.

Impact

Any admin action that disables a WHYPE-market collateral, which is normal risk management rather, permanently locks that collateral for every holder.

Code Snippet

https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/blob/4e11694f7a029115f3fdb01bd98b6425744c4bd3/delpho_contracts/src/periphery/DelphoRouterV2.sol#L68

https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/blob/4e11694f7a029115f3fdb01bd98b6425744c4bd3/delpho_contracts/src/periphery/DelphoRouterV2.sol#L74

https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/blob/4e11694f7a029115f3fdb01bd98b6425744c4bd3/delpho_contracts/src/periphery/DelphoRouterV2.sol#L133

https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/blob/4e11694f7a029115f3fdb01bd98b6425744c4bd3/delpho_contracts/src/periphery/DelphoRouter.sol#L389

Tool Used

Manual Review.

Recommendation

Restore V1's asymmetry: gate entry, never gate exit. Remove `_validateMarketTokens` and `_validateMarketToken` from `requestWithdrawal`, `cancelWithdrawalRequest` and `repayAndWithdraw`, keeping them on `depositCollateral` and `depositAndMint` where `_handleMarketDeposit` already applies them.

Issue M-2: PSM mints and redeems USDV against USDC at a hardcoded 1:1 with no oracle, so a USDC depeg directly under-backs USDV [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/issues/68

Summary

`DelphoPSM.mint` and `DelphoPSM.redeem` compute their output arithmetically from the input amount, applying only a fee. Neither consults an oracle:

```
usdvOut = amount - fee;        // mint
collateralOut = usdvAmount - fee;  // redeem
```

The rest of the protocol prices the same USDC through Chainlink, and the ConfigProvider already carries a stablecoin clamp for it. So the PSM is the only mint path in the system that will accept depegged collateral at face value, up to `psmMintCap` of 2,000,000 USDV.

Vulnerability Detail

USDV is 6 decimals and USDC is 6 decimals, so `amount - fee` is a unit-for-unit swap at exactly par regardless of market price. There is no price lookup, no deviation bound and no circuit breaker in either direction.

The CDP path does the opposite. It values USDC through `DelphoOracle.getUsdValuesStrict`, and `_getTokenPriceStrict` applies the configured stablecoin clamp:

```
if (bounds.isStable && bounds.maxPrice != 0 && price > bounds.maxPrice) {
    price = bounds.maxPrice;
}
```

On the live provider `tokenPriceBounds(USDC)` is `(1000000, true)`, so the CDP already treats USDC as at most one dollar while marking it down below par. The PSM ignores both halves of that.

The result is direct over-issuance. If USDC trades at 0.95 dollars, anyone can buy it on the open market and mint USDV from the PSM at par minus the 10 basis point mint fee. The PSM's recorded liability, `psmLiabilities[collateral]`, then exceeds the market value of the collateral backing it by the depeg amount, and USDV becomes under-backed by that difference across the whole minted volume. No attacker role, no privileged access and no unusual sequencing is required: `Router.psmMint` is permissionless.

The exposure is bounded by `psmMintCap`, which the initializer sets to 2,000,000 USDV, and by the mint fee. A one percent depeg with the cap fully used is roughly 20,000 dollars of

unbacked USDV. Redemption is exposed symmetrically: at 0.95 dollars a redeemer receives a full unit of USDC per USDV, draining the liquid buffer faster in value terms than the liability it retires.

Note that `TokenPriceBounds` carries only `maxPrice` and no floor, so the existing clamp cannot express the redemption-side protection even if the PSM used the oracle today.

Impact

Any USDC depeg beyond the mint fee lets anyone convert discounted collateral into full-value USDV, degrading USDV's backing for every holder up to the mint cap. Historic USDC depegs have exceeded the 10 basis point mint fee by orders of magnitude, so the fee is not a meaningful defence. The same asymmetry lets redeemers extract full-unit collateral for depegged-value USDV, accelerating buffer depletion precisely when the peg is under stress.

The protocol already owns the machinery to prevent this and applies it on the CDP path, so this is an inconsistency between two mint paths for the same asset rather than a missing capability.

Code Snippet

https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/blob/4e11694f7a029115f3fdb01bd98b6425744c4bd3/delpho_contracts/src/core/DelphoPSM.sol#L151

https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/blob/4e11694f7a029115f3fdb01bd98b6425744c4bd3/delpho_contracts/src/core/DelphoPSM.sol#L215

https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/blob/4e11694f7a029115f3fdb01bd98b6425744c4bd3/delpho_contracts/src/core/DelphoOracle.sol#L182

https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/blob/4e11694f7a029115f3fdb01bd98b6425744c4bd3/delpho_contracts/src/configurators/base/interfaces/IDelphoOracleConfig.sol#L22

Tool Used

Manual Review.

Recommendation

Price both PSM legs through the oracle and clamp each direction so it favours the protocol:

- **Mint:** use `min(oraclePrice, 1e6)`. Depegged collateral mints proportionally less USDV, and a premium above par never mints more than par. This needs no new configuration, because `getTokenPriceStrict` already returns exactly this for USDC given the live `tokenPriceBounds(USDC)` of `(1000000, true)`.
- **Redeem:** use `max(oraclePrice, 1e6)`. A redeemer never receives more collateral units than par during a depeg, and never receives premium collateral cheaply. This requires a floor, so add a `minPrice` field to `TokenPriceBounds` or apply the bound in the PSM.

Consider also pausing `psmMint` automatically when the oracle price deviates beyond a configured threshold, using the existing per-collateral `psmMintPaused` mapping. Note that routing through `getTokenPriceStrict` also inherits its staleness revert, which is the desired behaviour: the PSM should not mint against an unknown price.

Issue M-3: PSM min single redemption strands dust [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/issues/74

Summary

`DelphoPSM::redeem()` enforces a floor on the size of a single redemption but never checks the residual the redemption leaves behind, so any redemption that drops `globalBufferFunds[collateral]` (or `psmLiabilities[collateral]`) to a nonzero value below `psmMinSingleRedemption` strands that residual permanently and bricks the entire PSM redemption path for that collateral until an admin reallocates funds or new collateral is minted in.

Vulnerability Detail

`DelphoPSM::redeem()` rejects anything smaller than the configured floor before doing any other work: DelphoPSM.sol#L195.

A valid redemption therefore has to satisfy `psmMinSingleRedemption <= usdvAmount <= min(psmLiabilities[collateral], globalBufferFunds[collateral])`. Nothing in the function constrains what the subtraction leaves behind, and no code path anywhere in the contract exempts a final sweep from the floor. The moment `min(psmLiabilities[collateral], globalBufferFunds[collateral])` lands strictly between zero and `psmMinSingleRedemption`, the feasible interval is empty and every possible redemption reverts: a large enough amount fails the balance checks with `DPSM_INSUFFICIENT_LIQUID_RESERVE` or `DPSM_INSUFFICIENT_LIABILITIES`, and a small enough amount fails the floor with `DPSM_REDEMPTION_BELOW_MIN`.

Impact

The redemption side of the peg stability module can be shut down for a collateral by a single ordinary looking redemption, removing the peg floor for every USDV holder until an admin intervenes. Residual collateral below `psmMinSingleRedemption` in the buffer book is unredeemable through the PSM entirely. The griefing cost is only the redeem fee on the attacker's own redemption, and the state is trivially re established after each recovery, so the denial of service is cheap and repeatable.

Code Snippet

DelphoPSM.sol#L193-L197 and DelphoPSM.sol#L220-L234

```
if (usdvAmount == 0) revert DPSM_ZERO_AMOUNT();
```

```
if (usdvAmount < psmMinSingleRedemption) revert DPSM_REDEMPTION_BELOW_MIN();
if (usdvAmount > psmMaxSingleRedemption) revert DPSM_REDEMPTION_ABOVE_MAX();

...

uint256 fee = (usdvAmount * cfg.psmRedeemFee()) / BPS;
collateralOut = usdvAmount - fee;

if (collateralOut < minCollateralOut) revert DPSM_SLIPPAGE_EXCEEDED(collateralOut,
↪  minCollateralOut);
if (usdvAmount > psmLiabilities[collateral]) revert DPSM_INSUFFICIENT_LIABILITIES();
if (usdvAmount > globalBufferFunds[collateral]) revert
↪  DPSM_INSUFFICIENT_LIQUID_RESERVE();

// update storage
psmStartRedeemableTimestamp[user][collateral] = block.timestamp;
psmDailyRedemptions[user][collateral] = newUserTotal;
psmGlobalDailyRedemptions[collateral] = newGlobalTotal;

IDelphoStable(usdv).burn(address(this), usdvAmount);
psmLiabilities[collateral] -= usdvAmount;
globalBufferFunds[collateral] -= usdvAmount;
```

Tool Used

Manual Review

Recommendation

Make the floor aware of the remaining redeemable balance. Exempt a redemption that clears the whole remaining book, and reject any redemption that would leave a nonzero residual below the floor so the dust state can never be created in the first place. The floor check has to move below the balance checks so it can read the current books.

Issue L-1: A system address must never be configured for USDC, because both plausible values silently destroy funds [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/issues/69

Summary

USDC has no HyperCore system-address bridge route. Its only valid path to Core is `coreDepositWalletUsdc().deposit(amount, destinationDex)`, and `DelphoBaseExecutor._transferCollateralToCore` correctly routes it through a dedicated branch that never reads `systemAddress`.

The generic `else` branch protects against a mis-route with `if (systemAddress == address(0)) revert DEB_INVALID_SYSTEM_ADDRESS()`. Configuring any value for USDC disarms that guard, and both plausible values destroy funds: the initializer hardcodes the deposit-wallet contract, and the live provider holds a codeless address.

Vulnerability Detail

The repo's own vendored HyperEVM library is explicit. `CoreWriterLib.bridgeToCore` computes `getSystemAddress(token)` and then deliberately refuses to use it for USDC:

```
if (HLConstants.isUsdc(token)) {
    IERC20(HLConstants.usdc()).approve(address(coreDepositWallet), evmAmount);
    coreDepositWallet.deposit(evmAmount, uint32(type(uint32).max));
} else if (isHype(token)) { ... } else {
    IERC20(tokenAddress).safeTransfer(systemAddress, evmAmount);
}
```

`PrecompileLib.getTokenIndex` also short-circuits USDC before consulting the token registry, indicating USDC is not a registry-linked token with an EVM system-address route.

Both configurable values are unsafe as a `safeTransfer` destination:

- `0x2000000000000000000000000000000000000000`, the current live value, has **zero bytes of code**. A transfer succeeds at the ERC-20 level, nothing credits a HyperCore balance, and there is no owner or admin. Funds are permanently lost.
- `0x6B9E773128f453f5c2C60935Ee2DE2CBc5390A24`, the initializer default, is the deposit wallet itself: a transparent upgradeable proxy whose admin is not Delpho. A plain `safeTransfer` bumps its balance without invoking `deposit()`, so there is no Core credit, no event and no attribution. The funds merge into its balance and recovery requires Hyperliquid's cooperation.

The USDC branch is selected by `else if (token == cfg.stablecoinToken(StableCoinType.USDC))`. Anything that makes USDC miss that comparison, such as `stablecoinToken(USDC)` being unset on a partially configured provider or pointed at a different address, drops USDC into the generic branch, which then reads the configured system address and sends there.

Note this is not migration-specific: the live provider already returns `0x2000000000000000000000000000000000000000` for `systemAddress(USDC)`, so the guard is disarmed in production today.

Impact

The guard that exists to convert a USDC mis-route into a clean revert is disabled by configuration.

Code Snippet

https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/blob/4e11694f7a029115f3fdb01bd98b6425744c4bd3/delpho_contracts/src/configurators/base/DelphoHyperCoreConfig.sol#L52

https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/blob/4e11694f7a029115f3fdb01bd98b6425744c4bd3/delpho_contracts/src/executor/base/DelphoBaseExecutor.sol#L179

https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/blob/4e11694f7a029115f3fdb01bd98b6425744c4bd3/delpho_contracts/script/prod/upgrade/base/BaseConfigProviderUpgrade.s.sol#L307

https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/blob/4e11694f7a029115f3fdb01bd98b6425744c4bd3/delpho_contracts/script/prod/nonStable/02_DeploymentScript.s.sol#L165

Tool Used

Manual Review.

Recommendation

Leave `systemAddress(USDC)` at `address(0)` everywhere, so a mis-route reverts. Four write sites must stop setting it:

1. **`DelphoHyperCoreConfig` initializer:** delete the `_systemAddresses[USDC]` assignment and its `SystemAddressUpdated` event. Keep the USDT entry.
2. **`BaseConfigProviderUpgrade._syncStablecoinConfig`:** it is invoked for `StableCoinType.USDC` and calls `setSystemAddress(oldToken, oldSystemAddress)`, so it will copy

the live value onto the new provider. Skip the system-address half for USDC and sync only `stablecoinToken`. Its existing early return on `oldSystemAddress == address(0)` already cooperates once the value is zero upstream.

3. **`script/prod/nonStable/02_DeploymentScript`:** remove the `setSystemAddress(USDC, USDC_SYSTEM_ADDRESS)` call.
4. **`script/prod/stable/02_DeploymentScript`:** remove the equivalent call.

Issue L-2: WHYPE PM executor is granted the system-wide HANDLER_ROLE where no existing role is correctly scoped [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/issues/70

Summary

To let the new WHYPE-market executor pay yield into the shared `DelphoRewardDistributor`, the migration grants it `HANDLER_ROLE` on the stable ConfigProvider. That is the broadest hot role in the system: 28 `onlyHandler` entry points across every stable and deprecated executor, plus `swapAndDistribute`. It needs authority over exactly one function, `notifyRewardAmount`.

The obvious narrower substitute is also wrong. `EXECUTOR_ROLE` satisfies `DelphoPSM.onlyExecutor`, which would let a WHYPE-market executor pull USDC from the PSM and break the rollout's stated market isolation.

Vulnerability Detail

`DelphoRewardDistributor` resolves roles against its own provider, which after migration is the stable one, so a WHYPE-market executor needs a stable-provider role to pay in. The gate accepts either role:

```
require(config.isHandler(msg.sender)||config.isExecutor(msg.sender) ,
↪  DRD_UNAUTHORIZED());
```

`DelphoConfigProviderMigration.run()` and its Safe-calldata twin `_buildTxList()` satisfy it with the broader one:

```
IRoleAdmin(NEW_USDC_CONFIG_PROVIDER).grantRole(stableConfig.HANDLER_ROLE(),
↪  NEW_WHYPE_PM_EXECUTOR);
```

The intent is narrow: the assertion in `_postMigrationChecks()` is labelled "WHYPE PM reward role missing". But every executor's `onlyHandler` modifier resolves `isHandler` through the stable provider, so the grant confers authority over 28 functions including `clawBack`, `transferFundsToVault`, `executeCoreAction` and the swap extension.

Substituting `EXECUTOR_ROLE` trades one over-grant for a worse one. `DelphoPSM.onlyExecutor` resolves against the PSM's own provider, which is the stable one, and the USDC-only allowlist gates the *collateral*, not the *caller*. A WHYPE executor could then call `deployReserves(USDC, amount)` and pull USDC out of `fundsForExecutor`, violating the scope document's requirement that a WHYPE executor cannot pull from the PSM.

Not exploitable in the target commit. `NEW_WHYPE_PM_EXECUTOR` is a contract with no arbitrary-target call: `_sendCoreAction` forwards only to `cfg.coreWriterAddress()`, and `DelphoWhypePMExecutor` does not inherit `DelphoExecutorSwapExtension`. The authority is dormant until that executor gains a swap extension, an adapter callback, or any function forwarding caller-supplied calldata.

Impact

Standing authority over 28 stable-market handler entry points where one function is needed, on a live provider. Hot-role abuse is in scope per the trust model, unlike admin compromise. No funds are at risk today, hence Low, but the blast radius is unnecessary, undocumented, cross-market, and one ordinary feature addition from being live. It also silently widens effective `HANDLER_ROLE` membership, which the hardcoded role replay and single-member post-migration checks will not surface.

Code Snippet

https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/blob/4e11694f7a029115f3fdb01bd98b6425744c4bd3/delpho_contracts/script/prod/upgrade/02_DelphoMigrate.s.sol#L197

https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/blob/4e11694f7a029115f3fdb01bd98b6425744c4bd3/delpho_contracts/src/DelphoRewardDistributor.sol#L38

https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/blob/4e11694f7a029115f3fdb01bd98b6425744c4bd3/delpho_contracts/src/core/DelphoPSM.sol#L89

Tool Used

Manual Review

Recommendation

Add a dedicated `REWARD_NOTIFIER_ROLE` to `BaseConfig`, check it in `notifyRewardAmount`, and grant only that to each market's executors:

```
-require(config.isHandler(msg.sender)||config.isExecutor(msg.sender) ,
↪  DRD_UNAUTHORIZED());
+require(config.isRewardNotifier(msg.sender), DRD_UNAUTHORIZED());
```

This needs no storage changes on either side: role identifiers are compile-time constants, both providers are fresh deployments, and the distributor's layout is unchanged. It does add a fourth live-proxy upgrade, since `DelphoRewardDistributor` is currently only

repointed, so sequence the upgrade in the same batch as the grant or yield forwarding reverts in the interval.

If that upgrade cannot be accommodated, keep `HANDLER_ROLE` as an accepted risk, document it in the scope document's trust model, and add a standing rule that `DelphoWhypePMExecutor` must never gain an arbitrary-target call, since that property is the only thing keeping the grant inert.

Issue L-3: DelphoRouterV2 re-implements six V1 functions that are already reusable, four needing only a visibility change and two needing nothing at all [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/issues/71

Summary

Every helper in `DelphoRouter` is declared `private`, which in Solidity is not accessible to a derived contract. `DelphoRouterV2` inherits `DelphoRouter` but consequently cannot call any of them, so it re-implements four internal helpers. It additionally duplicates two `external` self-call wrappers that were already inherited and callable exactly as they stand.

All of the V1 originals are market-agnostic: they take their dependencies as parameters and reference no ConfigProvider. Changing `private` to `internal` on four of them, and simply deleting the two redundant wrappers with no behavioural change.

Vulnerability Detail

There is no behavioural difference. The one apparent exception is not one: V1's `if (usdvToMint == 0) return;` is dead code, because the preceding `if (debt >= targetDebt) return;` already guarantees `usdvToMint >= 1`.

The four internal helpers are reusable because each receives everything it needs as an argument. `_deLeverageUser` and `_reLeverageUser` take a `DelphoContracts` struct plus the caps, `_handleRemainingCollateralAfterRedemption` takes the vault, and `_validateLengths` is `pure`. None touches `delphoConfigProvider` or calls `_configProvider()`, so none is bound to a single market.

The two external wrappers require no change at all. Both V1 wrappers are `external` and therefore inherited, so `this.deLeverageUserExternal` is directly callable from V2, and the `msg.sender != address(this)` guard is satisfied by V2's own self-call because both resolve to the same proxy address. Neither wrapper is declared in `IDelphoRouter` or `IDelphoRouterV2`, so deleting the V2 copies changes no interface and no `interfaceId`.

`_handleDeposit` is the sole genuine exception and illustrates the pattern. It resolves the provider internally with `_configProvider()`, hardcoding the default market, which is why V2 needed a separate `_handleMarketDeposit` that accepts a provider argument. Unifying that one requires a signature change, not just a visibility change.

Impact

Duplicated code.

Code Snippet

https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/blob/4e11694f7a029115f3fdb01bd98b6425744c4bd3/delpho_contracts/src/periphery/DelphoRouterV2.sol#L320

https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/blob/4e11694f7a029115f3fdb01bd98b6425744c4bd3/delpho_contracts/src/periphery/DelphoRouterV2.sol#L243

https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/blob/4e11694f7a029115f3fdb01bd98b6425744c4bd3/delpho_contracts/src/periphery/DelphoRouterV2.sol#L204

https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/blob/4e11694f7a029115f3fdb01bd98b6425744c4bd3/delpho_contracts/src/periphery/DelphoRouter.sol#L491

Tool Used

Manual Review

Recommendation

First delete both V2 external wrappers and retarget the self-calls at the inherited V1 ones. This needs no other change:

```
-address(this).call(abi.encodeCall(this.deLeverageUserV2External, (users[i],
↪  contracts, config.maxLtvBps(), config.minMint())));
+address(this).call(abi.encodeCall(this.deLeverageUserExternal,   (users[i],
↪  contracts, config.maxLtvBps(), config.minMint())));
```

Then change the four V1 internal helpers from `private` to `internal` and delete the V2 copies:

```
-function _validateLengths(address[] calldata tokens, uint256[] calldata amounts)
↪  private pure returns (uint256 len) {
+function _validateLengths(address[] calldata tokens, uint256[] calldata amounts)
↪  internal pure returns (uint256 len) {
```

Apply the same to `_handleRemainingCollateralAfterRedemption`, `_deLeverageUser` and `_reLeverageUser`.

Issue L-4: PSM documented USDT but unsupported [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/issues/72

Summary

`DelphoPSM` documents USDT as a supported PSM collateral, but `DelphoPSM::_isSupportedCollateral()` resolves only `StableCoinType.USDC` from the config provider, so USDT can never be minted against or deployed even though it is a fully configured `stablecoinToken`.

Vulnerability Detail

The contract level NatSpec states that supported collateral tokens are resolved via the configured `stablecoinToken` mappings and names both USDC and USDT as the supported set: DelphoPSM.sol#L13-L20.

The actual acceptance check reads a single entry of that mapping. `DelphoPSM::_isSupportedCollateral()` fetches `cfg.stablecoinToken(StableCoinType.USDC)` and returns true only when the supplied address equals it: DelphoPSM.sol#L379-L386. The USDT branch of the mapping is never consulted.

The config provider does populate USDT. `DelphoHyperCoreConfig` initializes `stablecoinToken[StableCoinType.USDT]` to a live token address alongside USDC, and exposes `setStablecoinToken()` so an admin can repoint either entry: DelphoHyperCoreConfig.sol#L48-L49. The `StableCoinType` enum likewise carries a distinct `USDT` member: DelphoExecutorTypes.sol#L26-L30.

Consequently `DelphoPSM::mint()` reverts with `DPSM_UNSUPPORTED_COLLATERAL` for USDT (DelphoPSM.sol#L147) and `DelphoPSM::deployReserves()` reverts for the same reason (DelphoPSM.sol#L253-L255). Note that `DelphoPSM::redeem()` never calls `_isSupportedCollateral()` at all, so the mint and redeem sides do not even agree on what "supported" means.

Impact

The deployed PSM is single collateral in practice while its documentation, its config surface and its enum all advertise two. Integrators and off chain tooling that trust the NatSpec will build USDT mint flows that revert unconditionally. Because the accepted asset is baked into an internal function rather than read from configuration, listing USDT later requires a UUPS upgrade of `DelphoPSM` rather than an admin call to `setStablecoinToken()`, which is a materially heavier and riskier operation than the configuration driven design the comments describe. No funds are at risk, so this is Low.

Code Snippet

DelphoPSM.sol#L13-L20 and DelphoPSM.sol#L379-L386

```
/**
 * @title DelphoPSM
 * @notice Peg Stability Module (PSM) enabling two-way par swaps between supported
 ↪  stablecoin collateral and USDV.
 * @dev UUPS upgradeable contract backed by multi-collateral stablecoin reserves.
 ↪  Supported collateral tokens are
 *      resolved via configured `stablecoinToken` mappings (USDC, USDT).
 ↪  Mint/redeem fees
 *      are sent immediately to `feeReceiver`. Executor returns are capped at
 ↪  deployed principal (partial returns allowed).
 *      Liabilities are tracked per collateral; redeem is bound to that
 ↪  collateral's liabilities and buffer.
 */

    /// @dev Returns true when `collateral` matches a non-zero configured
    ↪  `stablecoinToken` for USDC.
    function _isSupportedCollateral(address collateral) internal view returns
    ↪  (bool) {
        if (collateral == address(0)) return false;

        IDelphoConfigProvider cfg = IDelphoConfigProvider(delphoConfigProvider);
        address usdc = cfg.stablecoinToken(StableCoinType.USDC);
        return usdc != address(0) && collateral == usdc;
    }
```

Tool Used

Manual Review

Recommendation

Either accept every configured stablecoin type, or narrow the documentation to the asset that is actually supported. The configuration driven fix:

```
     /// @dev Returns true when `collateral` matches a non-zero configured
     ↪  `stablecoinToken`.
     function _isSupportedCollateral(address collateral) internal view returns
     ↪  (bool) {
         if (collateral == address(0)) return false;

         IDelphoConfigProvider cfg = IDelphoConfigProvider(delphoConfigProvider);
-        address usdc = cfg.stablecoinToken(StableCoinType.USDC);
-        return usdc != address(0) && collateral == usdc;
```

```
+        if (collateral == cfg.stablecoinToken(StableCoinType.USDC)) return true;
+        if (collateral == cfg.stablecoinToken(StableCoinType.USDT)) return true;
+        return false;
     }
```

If single collateral operation is the intended design, remove the USDT reference from the contract NatSpec instead, and apply the same check in `DelphoPSM::redeem()` so both legs share one definition of supported collateral.

Issue L-5: PSM mint does not open redemption window [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/issues/73

Summary

`DelphoPSM::mint()` credits `psmLiabilities` and `globalBufferFunds` but touches none of the redemption window state that `DelphoPSM::redeem()` maintains, so the two legs of a PSM round trip are accounted asymmetrically: minting creates redeemable liabilities without opening, resetting or crediting any redemption window, while every redemption permanently consumes the global per window budget.

Vulnerability Detail

`DelphoPSM::redeem()` is rate limited by two independent accumulators. The per user leg reads `psmStartRedeemableTimestamp[user][collateral]`, treats the accumulator as live only while the elapsed time is under `psmWindowDuration`, and checks the running total against `psmMaxDailyRedemption`: DelphoPSM.sol#L199-L207. The global leg is a tumbling window keyed by `psmGlobalRedemptionWindowStart[collateral]` and accumulated into `psmGlobalDailyRedemptions[collateral]` against `psmMaxGlobalDailyRedemption`: DelphoPSM.sol#L209-L218. Both are written on the way out, and the user window start is refreshed to `block.timestamp` on every single redemption: DelphoPSM.sol#L227-L230.

`DelphoPSM::mint()` updates only the books. It raises `psmLiabilities[collateral]` against `psmMintCap` (DelphoPSM.sol#L159-L161) and splits the net amount into `globalBufferFunds` and `fundsForExecutor` (DelphoPSM.sol#L163-L165). It never writes `psmStartRedeemableTimestamp`, `psmDailyRedemptions`, `psmGlobalRedemptionWindowStart` or `psmGlobalDailyRedemptions`, and there is no mint side cap other than the cumulative `psmMintCap`: DelphoPSM.sol#L137-L171.

Two concrete consequences follow.

First, a mint and redeem round trip is net destructive to redemption capacity. Fresh collateral entering through `DelphoPSM::mint()` restores the buffer that a redemption drained, but nothing gives back the `psmGlobalDailyRedemptions[collateral]` that the redemption consumed. An address that deposits its own collateral and immediately redeems the resulting USDV burns global budget that legitimate redeemers can no longer use for the remainder of the window, at a cost of only the mint fee plus the redeem fee, ten and twenty five basis points under the deployed defaults (DelphoPSMConfig.sol#L19-L29). The per user cap does not stop this, since the redemption accumulators are keyed by `msg.sender` forwarded from `DelphoRouter::psmRedeem()` (DelphoRouter.sol#L215-L230) and any number of fresh addresses can be used.

Second, an honest user who has exhausted their per user allowance and then deposits

new collateral cannot get it back out. Their newly minted USDV is fully backed by the liabilities and buffer they just created, but `psmDailyRedemptions[user][collateral]` is untouched by the mint and `psmStartRedeemableTimestamp[user][collateral]` was pushed forward to the timestamp of their last redemption, so the wait runs a full `psmWindowDuration` from that redemption rather than from the start of any window the mint might have opened.

Impact

Redemption capacity for a collateral can be exhausted for the rest of a window by an attacker who supplies their own collateral, at a cost bounded by the combined mint and redeem fees on the amount cycled, denying other USDV holders the PSM peg floor until `psmGlobalRedemptionWindowStart[collateral]` rolls. No funds are lost and the attacker pays fees to the protocol on every cycle, so the realistic outcome is temporary griefing of the redemption path plus the accounting inconsistency that a user's own fresh deposit does not extend their own redemption allowance. Low.

Code Snippet

DelphoPSM.sol#L152-L170

```
uint256 fee = (amount * cfg.psmMintFee()) / BPS;
usdvOut = amount - fee;

if (usdvOut < minUsdvOut) revert DPSM_SLIPPAGE_EXCEEDED(usdvOut, minUsdvOut);

uint256 newLiab = psmLiabilities[collateral] + usdvOut;
if (newLiab > cfg.psmMintCap()) revert DPSM_CAP_EXCEEDED();
psmLiabilities[collateral] = newLiab;

uint256 bufferShare = (usdvOut * cfg.psmBufferRatioBps()) / BPS;
globalBufferFunds[collateral] += bufferShare;
fundsForExecutor[collateral] += usdvOut - bufferShare;

_transferFeeToFeeReceiver(cfg, collateral, fee);
IDelphoStable(usdv).mint(user, usdvOut);
```

Tool Used

Manual Review

Recommendation

Make the two legs symmetric by rate limiting the mint side as well, so that recycling collateral through the PSM is bounded on entry and cannot be used to burn the

redemption budget of other holders.

Issue L-6: PSM daily limit bypass via multiple wallets [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/issues/75

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`DelphoPSM::redeem()` enforces its rolling redemption cap against the `user` address it is handed, and that address is simply the externally owned account that called `DelphoRouter::psmRedeem()`. Because creating a fresh account is free and permissionless, `psmMaxDailyRedemption` constrains nobody who is willing to split a redemption across several addresses, and the only cap that actually binds is the global tumbling window.

Vulnerability Detail

The per user window is keyed on the `user` argument in two mappings, `psmStartRedeemableTimestamp[user][collateral]` and `psmDailyRedemptions[user][collateral]`, both declared in DelphoPSM.

`DelphoPSM::redeem()` reads the accumulator for that address, adds the current amount and compares it against `psmMaxDailyRedemption`, here. A caller who has never redeemed has `prevUserStart == 0`, so `userWindowUsed` stays zero and the whole daily allowance is available to them immediately.

The `user` value is not an attested identity. `DelphoPSM::redeem()` only requires that the caller is the configured router, here, and `DelphoRouter::psmRedeem()` forwards its own `msg.sender` straight through as `user`, here. So the key is nothing more than whichever address funded and signed the transaction, and an actor holding USDV can distribute it to N fresh addresses and obtain N times the intended per address allowance in the same block.

Impact

The protection intended by `psmMaxDailyRedemption` does not exist against any determined redeemer. A single actor can drain up to the full `psmMaxGlobalDailyRedemption` and up to the whole `globalBufferFunds[collateral]` book within one window by fanning out across addresses, exhausting the redemption buffer that honest users share and forcing the remainder of the window to revert with `DPSM_INSUFFICIENT_LIQUID_RESERVE`. Severity is Low because the global window and the buffer balance still bound the total outflow, and no funds leave at an incorrect rate.

Code Snippet

DelphoPSM::redeem()

```
// sliding per-user window
uint256 userWindowUsed;
uint256 prevUserStart = psmStartRedeemableTimestamp[user][collateral];
if (prevUserStart != 0 && block.timestamp >= prevUserStart && block.timestamp -
↪  prevUserStart < psmWindowDuration) {
    userWindowUsed = psmDailyRedemptions[user][collateral];
}
uint256 newUserTotal = userWindowUsed + usdvAmount;
if (newUserTotal > psmMaxDailyRedemption) revert DPSM_DAILY_LIMIT_EXCEEDED();
```

DelphoRouter::psmRedeem()

```
function psmRedeem(address collateral, uint256 usdvAmount, uint256 minCollateralOut)
    external
    override
    nonReentrant
    whenNotPaused
    returns (uint256 collateralOut)
{

    IDelphoConfigProvider configProvider = _configProvider();
    address psm = configProvider.delphoPSM();
    if (psm == address(0)) revert DR_INVALID_ADDRESS();

    address delphoStable = configProvider.delphoStable();
    IERC20(delphoStable).safeTransferFrom(msg.sender, psm, usdvAmount);
    collateralOut = IDelphoPSM(psm).redeem(msg.sender, collateral, usdvAmount,
    ↪  minCollateralOut);
}
```

Tool Used

Manual Review

Recommendation

An address keyed cap cannot be made meaningful on a permissionless chain, so do not size risk against it. Either treat the global tumbling window as the real control and calibrate `psmMaxGlobalDailyRedemption` on the assumption that one actor can consume all of it, or gate PSM redemption on an allowlist or attested identity so that the key being rate limited is not free to mint.

```
+    /// @notice Addresses cleared to redeem through the PSM.
+    mapping(address user => bool) public psmRedeemAllowed;
+
+    function setPsmRedeemAllowed(address user, bool allowed) external onlyAdmin {
+        psmRedeemAllowed[user] = allowed;
+        emit PSMRedeemAllowlistUpdated(user, allowed);
+    }
+
     function redeem(address user, address collateral, uint256 usdvAmount, uint256
     ↪  minCollateralOut)
         external
         override
         whenNotPaused
         nonReentrant
         returns (uint256 collateralOut)
     {
         IDelphoConfigProvider cfg = IDelphoConfigProvider(delphoConfigProvider);
         if (msg.sender != cfg.delphoRouter()) revert DPSM_NOT_ROUTER();
         if (user == address(0)) revert DPSM_INVALID_ADDRESS();
+        if (!psmRedeemAllowed[user]) revert DPSM_NOT_ALLOWLISTED();
```

If an allowlist is not acceptable for the product, drop `psmMaxDailyRedemption` from the threat model entirely and document that the global window plus `globalBufferFunds[collateral]` are the only enforced bounds, so that operators do not size the global window on the false assumption that individual redeemers are capped.

Issue L-7: Redemption ignores per market health [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/issues/77

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`DelphoRouterV2::redeem()` resolves the target market purely from the caller supplied `marketKey`, and each `DelphoStableManager::redeem()` instance can only see its own market, so nothing steers redemption toward the weaker of the two markets that share the same USDV liability. Redemption is health aware inside a market and completely health blind between markets.

Vulnerability Detail

The multi market upgrade maps every collateral token to a market key and every market key to its own `DelphoConfigProvider`, which in turn owns its own vault, stable manager and sorted debt list (configureMarket). The production migration configures exactly two of them, `STABLE_MARKET` backed by USDC and USDT and `NON_STABLE_MARKET` backed by WHYPE and kHYPE, and wires the same `DelphoStable` token into both providers, so both markets mint and burn the same USDV (migration wiring).

Inside a single market the redemption ordering is exactly right. `DelphoStableManager::_redeemFromBorrowers()` snapshots borrowers starting from `IDelphoSortedDebts::getHead()`, and the list is maintained in ascending LCR order by `DelphoStableManager::_updateSortedDebt()`, so the head is the borrower closest to insolvency and redemption pressure lands on the weakest positions first (sorted debt walk, LCR ordering).

Between markets there is no such ordering, and no check of any kind. `DelphoRouterV2::redeem()` takes the market key from the caller and resolves it through `DelphoRouterV2::_marketByKey()`, which only verifies that a provider has been registered for that key (redeem, _marketByKey). The selected market's `DelphoStableManager::redeem()` then runs entirely against its own state and its own config, and validates only that the caller is the router (_validateCaller). It has no reference to the sibling market and no way to acquire one.

Every throttle that could have created balance is per instance as well. `startRedeemableTimestamp`, `dailyRedemptions`, `globalRedemptionWindowStart` and `globalDailyRedemptions` are all storage of one `DelphoStableManager`, and `StableManagerLib::checkAndUpdateDailyWindow()` charges them against that market's own `RedemptionConfig` (per market redemption budget). Exhausting the healthy market's daily allowance therefore does not push the next redeemer toward the unhealthy one, and the unhealthy market's untouched allowance exerts no pull either.

What a redeemer actually optimises for is the opposite of what the system needs. The market key chosen decides which collateral is received, and `StableManagerLib::selectCollateralProRata()` pays out pro rata in whatever that market holds (selectCollateralProRata). A redeemer will take USDC and USDT from the stables market, which is both the more liquid payout and the market whose LTV is least likely to be under stress, and will avoid WHYPE and kHYPE precisely when those prices are falling and the non stable market most needs its debt reduced. Each market's `redemptionFeeBps` is also independently configurable, so the cheaper market attracts flow regardless of which market is closer to `liquidationThresholdBps`.

Because both markets issue the same USDV, the two effects compound. Redemption steadily removes stable collateral and stable backed debt from the healthy side while the volatile side keeps its size, so the shared token's backing drifts toward the riskier market at exactly the times the safety valve was supposed to correct for.

Impact

No direct loss of funds and no attacker is required, this is the default outcome of rational redeemer behaviour. The consequence is structural imbalance: USDV is a single liability against two independent collateral pools, and the mechanism designed to shrink the riskiest debt is directed by redeemer preference rather than by system need. Over time redemption drains the most liquid, healthiest collateral first and leaves the shared stablecoin backed disproportionately by the weaker market, which raises the probability that the non stable market reaches liquidation, bad debt and fallback debt socialisation while the stables market has been shrinking the whole time.

Code Snippet

DelphoRouterV2::redeem()

```
function redeem(bytes32 marketKey, uint256 usdvAmount) external override
↪  nonReentrant whenNotPaused returns (uint256) {
    if (usdvAmount == 0) revert DR_INVALID_AMOUNT();
    IDelphoConfigProvider config = _marketByKey(marketKey);
    DelphoContracts memory contracts = config.getDelphoContracts();
    IStableManager stableManager = IStableManager(contracts.delphoStableManager);
    IDelphoVault vault = IDelphoVault(contracts.delphoVault);

    UserPosition memory loan =
    ↪  stableManager.getUserPosition(contracts.delphoStabilityPool);
    uint256 stabilityPoolAvailable = loan.active ? loan.usdvMinted : 0;
    uint256 normalRedeemAmount = FixedPointMathLib.min(stabilityPoolAvailable,
    ↪  usdvAmount);
    ...
    uint256 fallbackRedeemed = stableManager.redeem(msg.sender, usdvAmount,
    ↪  normalRedeemAmount);
    _requestRedeemedCollateral(vault, allTokens, collateralBefore);
```

```
    return normalRedeemAmount + fallbackRedeemed;
}
```

DelphoRouterV2::_marketByKey()

```
function _marketByKey(bytes32 marketKey) private view returns
↪   (IDelphoConfigProvider config) {
    address provider = marketConfigs[marketKey];
    if (provider == address(0)) revert DR_CONFIG_NOT_SET(marketKey);
    config = IDelphoConfigProvider(provider);
}
```

Tool Used

Manual Review

Recommendation

`DelphoRouterV2` already holds the registry of every configured market in `marketConfigs`, so it is the natural place to compare sibling health at redemption time. Track the configured market keys and, on redemption, read each market's aggregate LTV and either refuse or reprice a redemption against a market that is materially healthier than a sibling which is above its target LTV.

Alternatively, an intersected sorted debts can be used but such a redemption and liquidation would be much more complex due to dual design.

Issue L-8: Redeem incentive for unhealthy markets [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-08-delpho-update-aug-27th-2026/issues/78

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`DelphoStableManager::redeem()` credits the redeemer collateral worth exactly the net redeemed amount and charges a redemption fee on top of it, so a redemption is always value negative for the caller at peg. Redemption is the only permissionless action that deleverages a market, and it is the one action the protocol pays nobody to perform, so it does not happen when the market needs it most.

Vulnerability Detail

`DelphoStableManager::redeem()` splits the request into a stability pool portion and a fallback borrower portion, and both portions settle at a flat one to one USD exchange with a fee deducted from the redeemer's side (redeem).

On the pool path, `DelphoStableManager::_redeemFromPool()` computes the fee off the gross amount and reduces the seize budget by it (_redeemFromPool). The collateral actually handed over is selected by `StableManagerLib::selectCollateralProRata()`, which caps the seized USD value at the requested amount and never above it (selectCollateralProRata). The gross amount is then burned from the redeemer while only the net value of collateral is credited (burn and credit).

A redeemer burning 1 USDV therefore receives 0.995 USD of collateral, and that collateral is not even liquid on arrival: `DelphoRouterV2::redeem()` pushes the credited collateral through `requestWithdrawal`, so the portion beyond the vault buffer is queued to a future epoch and the redeemer carries price risk on it (_requestRedeemedCollateral).

Meanwhile the same transaction is unambiguously good for the market. Debt and collateral are both reduced by the same USD amount, so for any market whose LTV is above zero the ratio falls. A market holding 100,000 of collateral against 90,000 of debt sits at 90% LTV; a redemption of 30,000 leaves 70,000 of collateral against 60,000 of debt, which is 85.7% LTV, further from the liquidation threshold checked in `VaultMath::validateLiquidation()` (validateLiquidation) and from the borrow ceiling in `VaultMath::validateLtv()` (validateLtv). The same holds per position, and `DelphoStableManager::_redeemFromBorrowers()` already directs the pressure at the weakest borrowers by walking the sorted debt list from its head, which is the lowest LCR node (snapshot from getHead).

Impact

No direct loss of funds. The consequence is that the protocol's only permissionless deleveraging valve is dormant by construction. Deleveraging depends entirely on an external USDV discount that exceeds 0.5% plus gas, which is precisely the situation the deleveraging was meant to prevent. In the window where a market climbs from its target LTV toward `liquidationThresholdBps`, redemption would restore health cheaply and nothing pulls it in, so the market instead reaches the point where losses are realised through liquidation, bad debt accounting in `DelphoStableManager` and fallback debt socialisation across the remaining borrowers.

Code Snippet

DelphoStableManager::_redeemFromPool()

```
) private returns (uint256 consumed) {
    //   1. Fee & net redemption amount
    uint256 fee = _usdvAmount.mulDivUp(_redemptionFeeBps, MAX_BPS);
    uint256 netRedeem = _usdvAmount - fee;
    // Only reachable if redemptionFeeBps == MAX_BPS; kept as a safety net.
    if (netRedeem == 0) revert DSM_INVALID_AMOUNT();
    ...
    (address[] memory redeemTokens, uint256[] memory redeemAmounts,) =
        StableManagerLib.selectCollateralProRata(vault.getUserCollateralDetails(_st
        ↪  abilityPool), netRedeem);

    //   4. Reduce SP principal debt
    (uint256 newMinted,,,) = StableManagerLib.processRepayment(spPos.usdvMinted, 0,
    ↪  netRedeem);
    spPos.usdvMinted = newMinted;
    spPos.lastUpdatedAt = uint48(block.timestamp);

    //   5. Burn USDV from redeemer (CEI: debit user before crediting collateral)
    IDelphoStable(_usdv).burn(_user, _usdvAmount);

    //   6. Seize/credit collateral
    vault.debitCollateral(_stabilityPool, redeemTokens, redeemAmounts);
    vault.creditCollateral(_user, redeemTokens, redeemAmounts);
```

StableManagerLib::selectCollateralProRata()

```
function selectCollateralProRata(CollateralDetails memory collateralDetails,
↪  uint256 requiredUsd)
    internal
    view
    returns (address[] memory selectedTokens, uint256[] memory selectedAmounts,
    ↪  uint256 actualUsd)
{
```

```
    if (collateralDetails.totalValue == 0) return (selectedTokens, selectedAmounts,
    ↪  0);

    actualUsd = collateralDetails.totalValue.min(requiredUsd);
```

Tool Used

Manual Review

Recommendation

Pay the redeemer a bonus in collateral that scales with how far the market sits above its target LTV, keep the debt reduction at the net redeemed amount so the accounting is unchanged, and switch the bonus off near the liquidation threshold where handing out extra collateral would itself push the market down. The seize budget and the debt credit are already separate values in both redemption paths, so the change is local.

Apply the same treatment in `DelphoStableManager::_redeemFromBorrowers()` by widening `targetUsd` in `DelphoStableManager::_processOneFallbackBorrower()` while still crediting only `usdApplied` against the borrower's debt, and bound the bonus so that a single redemption can never raise the market LTV.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.